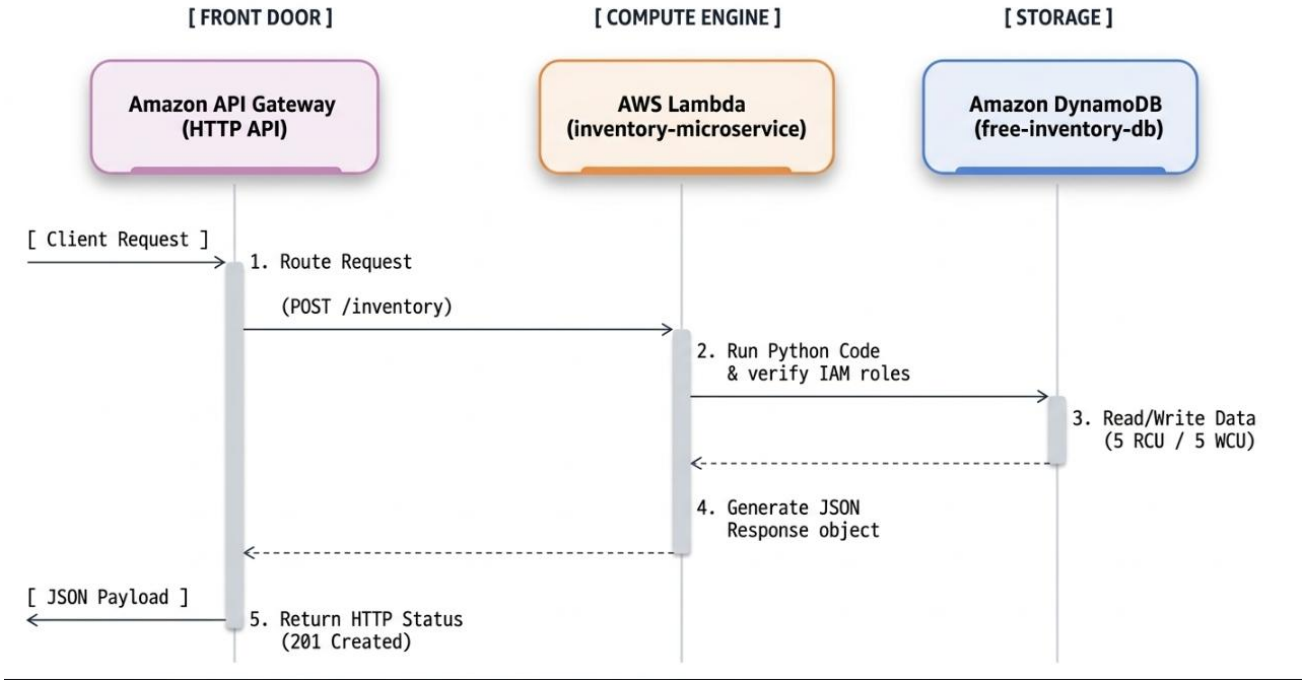
To build a 100% free microservices example, we will use a Serverless Architecture on AWS. This completely bypasses the cost of running virtual servers (EC2) or container clusters, keeping you safely within the AWS Free Tier.

We will build a miniature **"Product Inventory" system** consisting of a **single** microservice that allows you to add products and read them from a database.

STACK:

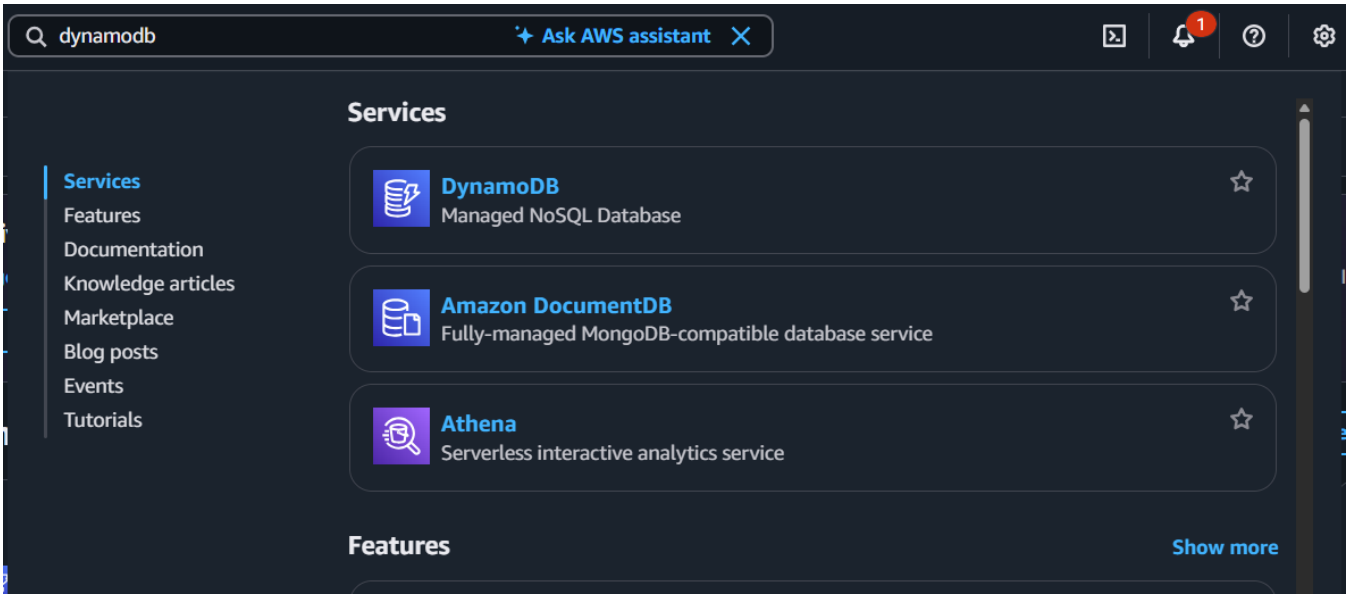
1. **AWS API Gateway**
2. **AWS Lambda**
3. **AWS DynamoDB**



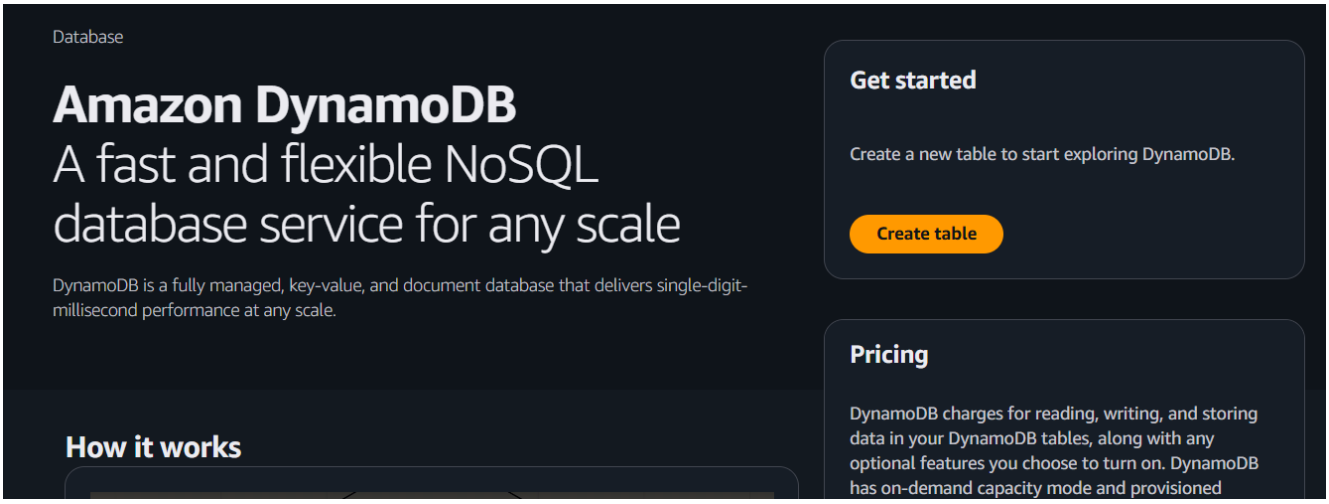
Step 1: Set Up the Free Database (DynamoDB)

We will create a database table using **Provisioned Capacity** to ensure it costs exactly $0.

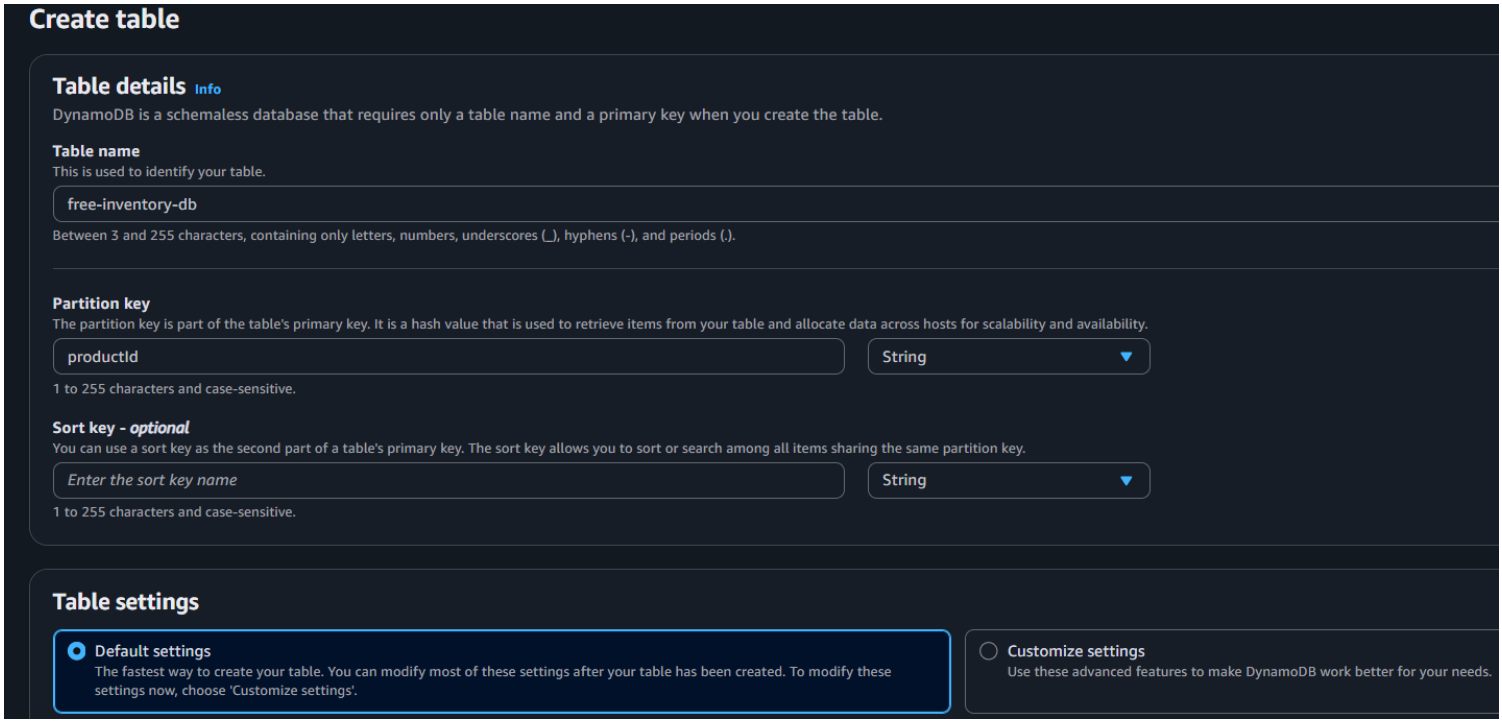
1. Log into your **AWS Management Console**.
2. Search for **DynamoDB** in the top search bar and open the service.



3. Click **Create table**.



4. Configure the following settings precisely:
    - **Table name:** free-inventory-db
    - **Partition key:** productId (Type: String)

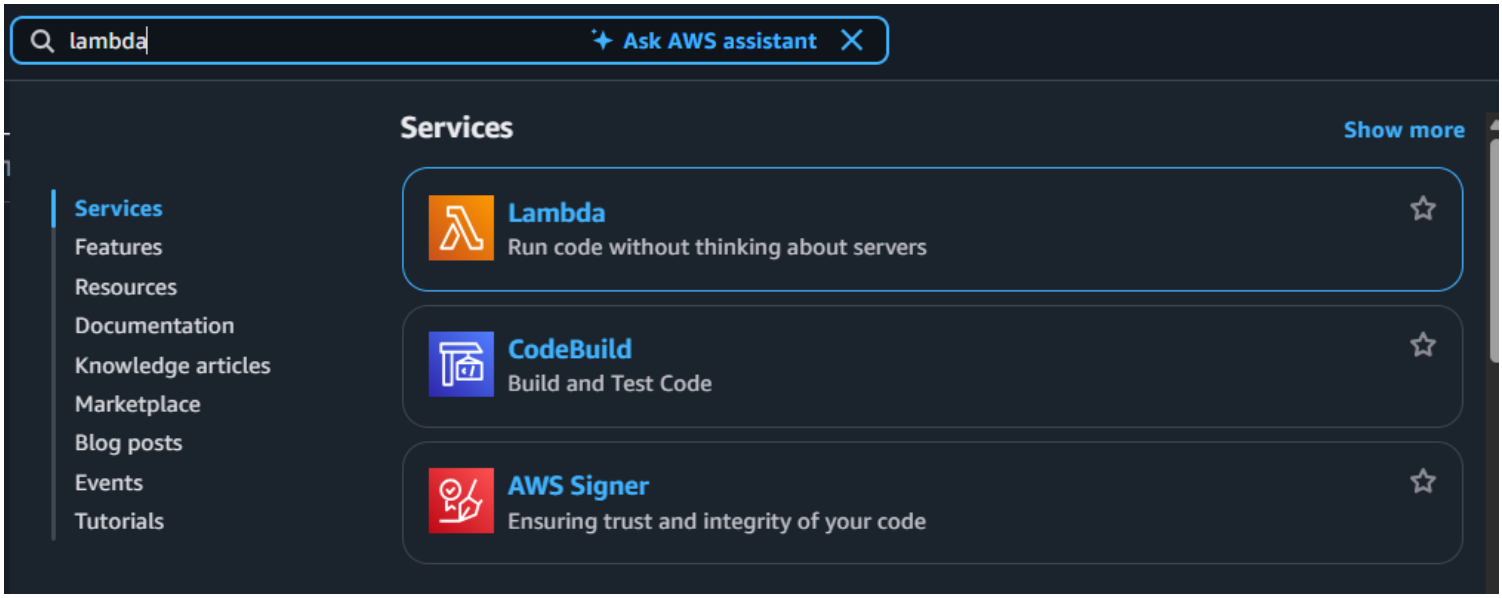


5. Scroll down to **Table settings** and change it from *Default* to **Customize settings**.
6. In the **Read/write capacity settings**:
    - Select **Provisioned** (Do *not* choose On-Demand).
    - Set **Read capacity auto-scaling** to **Off**.
    - Set **Write capacity auto-scaling** to **Off**.
    - Set **Read capacity units (RCU)** to 5 *(Free tier allows up to 25)*.
    - Set **Write capacity units (WCU)** to 5 *(Free tier allows up to 25)*.
7. Click **Create table**. (Takes 1 minute to get created)

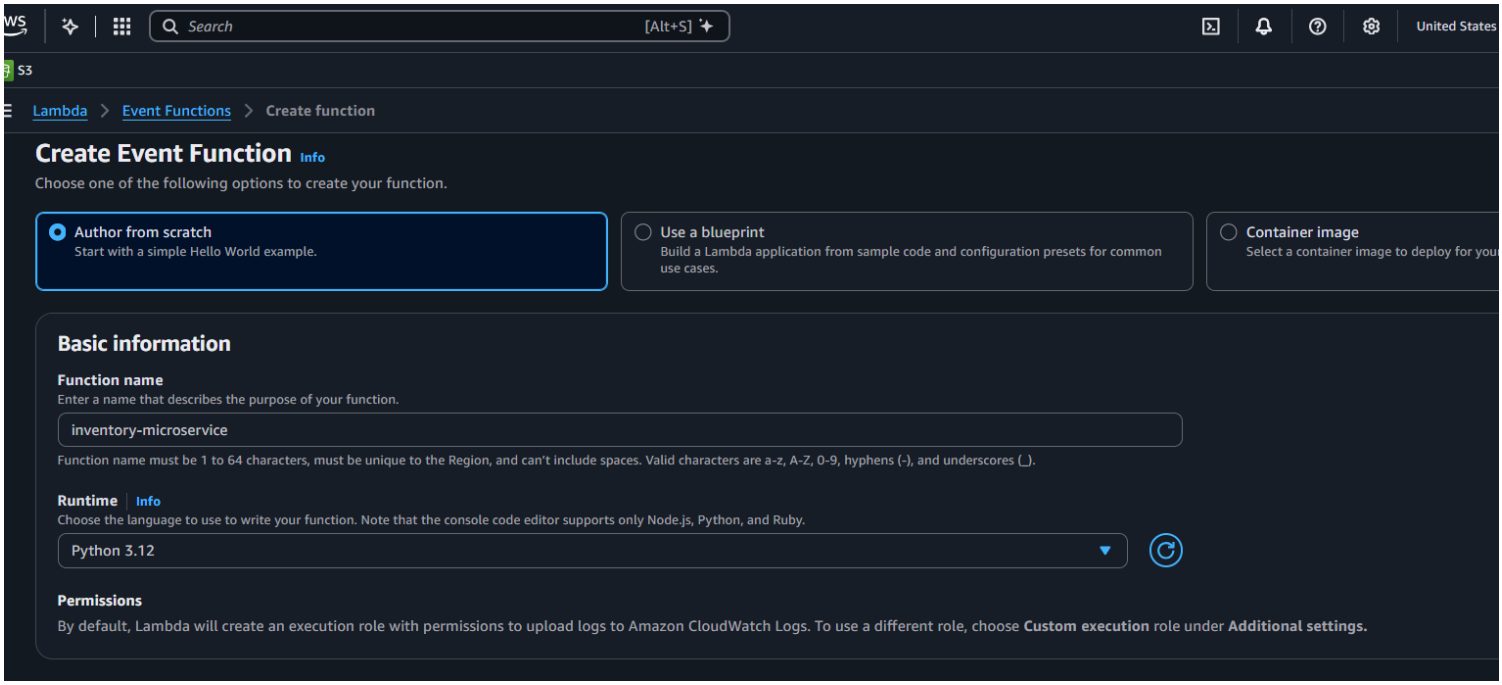
Step 2: Create the Microservice Logic (AWS Lambda)

Next, we will write the code that processes the microservice logic. AWS Lambda gives you **1 million free requests per month** forever.

1. Search for **Lambda** in the AWS Console search bar and open it.



2. Click **Create function**.
3. Choose **Author from scratch** and use these settings:
    - **Function name:** inventory-microservice
    - **Runtime:** Python 3.12 (or latest available version)



4. Click **Create function**.
5. Scroll down to the **Code source** section, open lambda_function.py, delete the placeholder code, and paste this complete microservice python script:

import json

import boto3

from boto3.dynamodb.conditions import Key

```
# Connect to our DynamoDB table
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('free-inventory-db')

def lambda_handler(event, context):
    # HTTP API (v2) stores the method inside requestContext -> http -> method
    http_method = event.get('requestContext', {}).get('http', {}).get('method')

    try:
        # Scenario 1: Add a product (POST request)
        if http_method == 'POST':
            body = json.loads(event.get('body') or '{}')
            table.put_item(Item={
                'productId': body['id'],
                'name': body['name'],
                'price': body['price']
            })
            return response(201, {"message": f"Product {body['name']} added successfully!"})

        # Scenario 2: Fetch a product (GET request)
        elif http_method == 'GET':
            query_params = event.get('queryStringParameters') or {}
            product_id = query_params.get('id')

            if not product_id:
                return response(400, {"error": "Missing 'id' query parameter"})

            res = table.get_item(Key={'productId': product_id})
            if 'Item' in res:
                return response(200, res['Item'])
            return response(404, {"error": "Product not found"})
```

```
        else:
            return response(405, {"error": f"Method {http_method} not allowed"})

    except Exception as e:
        return response(500, {"error": str(e)})

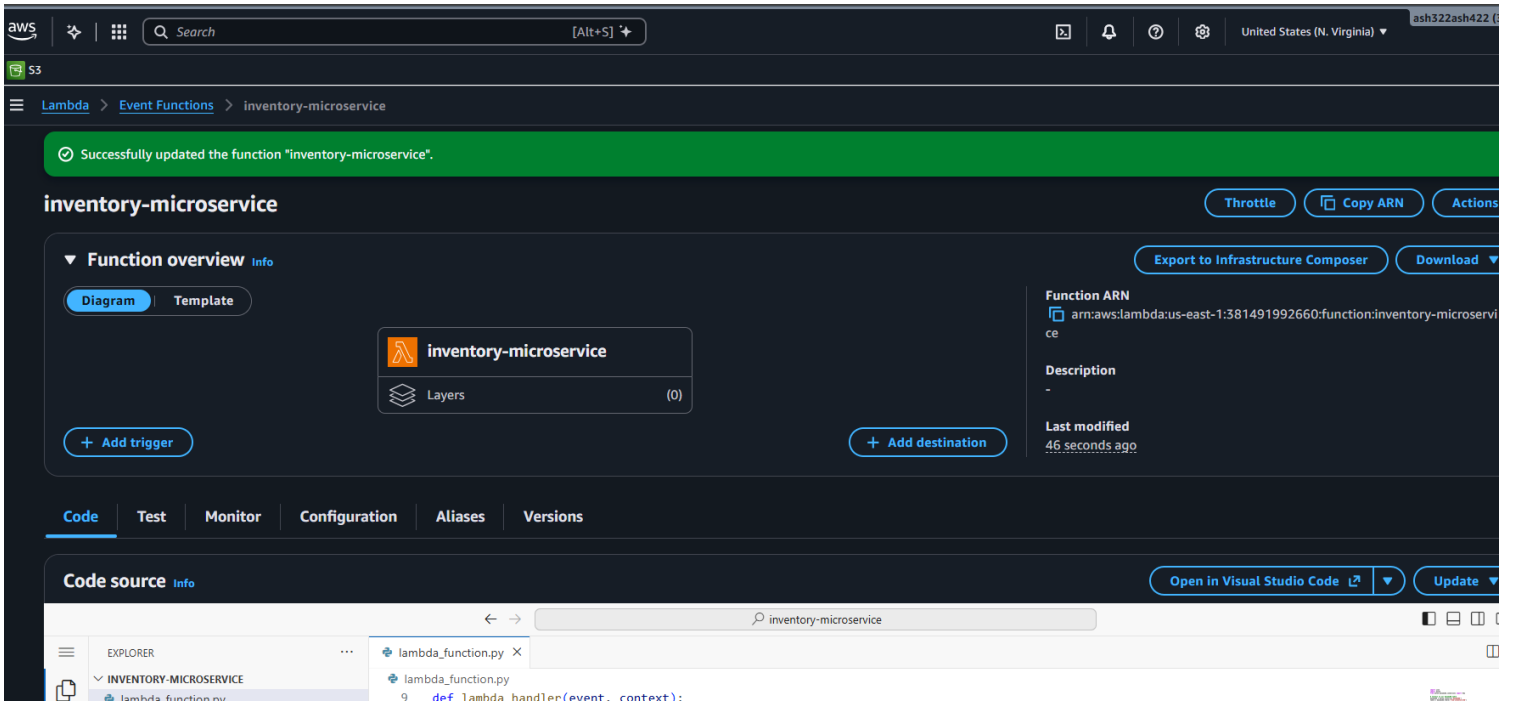
def response(status_code, body_content):
    """Helper function to format API Gateway responses"""
    return {
        "statusCode": status_code,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps(body_content)
    }
```

6. Click the **Deploy** button at the top of the code editor to save the changes. (Takes 1 minute)

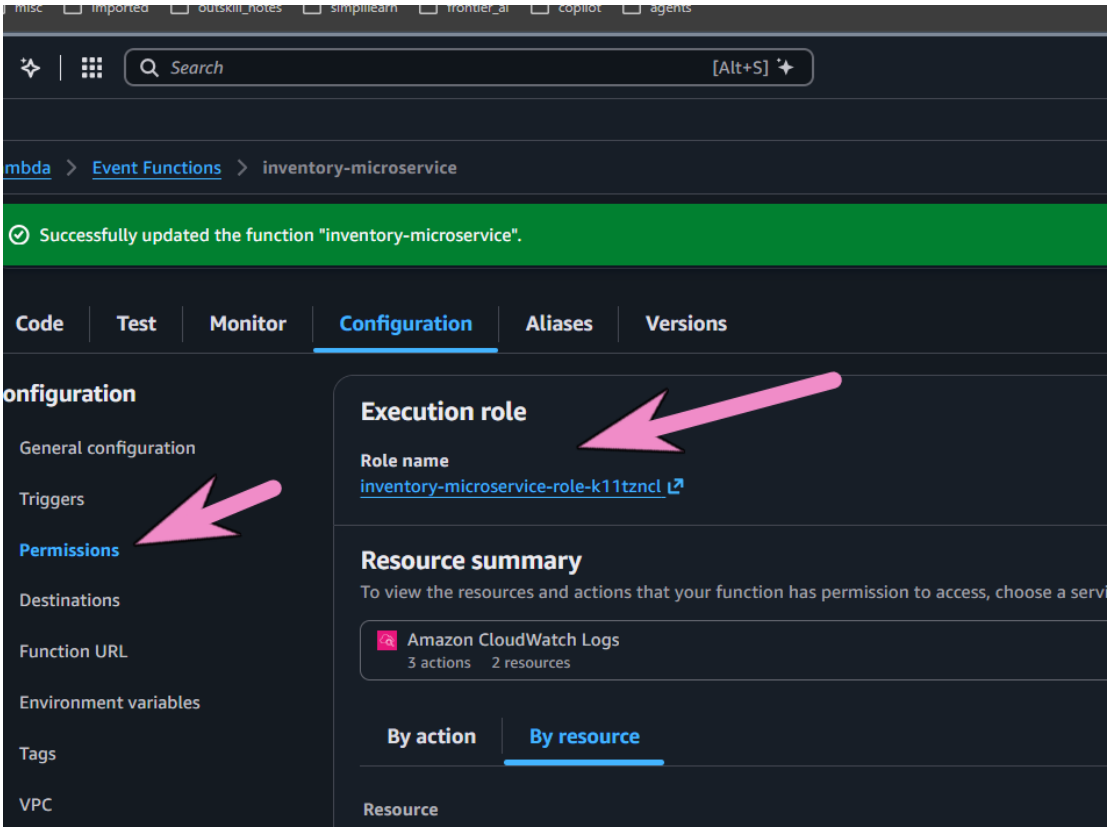
🔑 **Crucial Step: Give Lambda permission to touch DynamoDB**

By default, security rules block Lambda from reading or writing to your database.

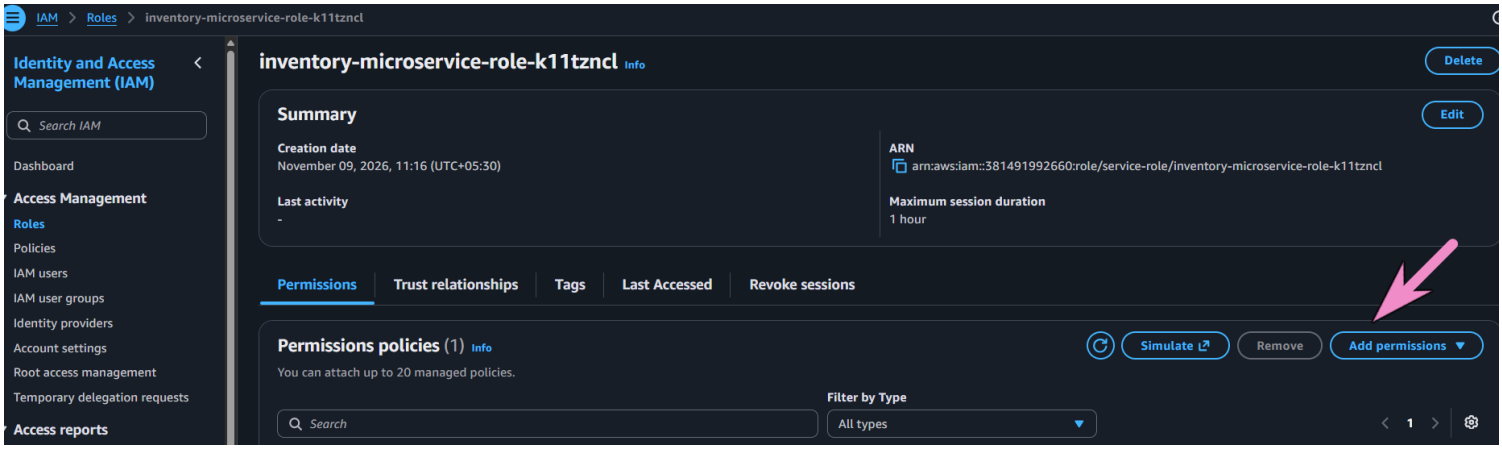
1. Within your Lambda function page, click the **Configuration** tab.



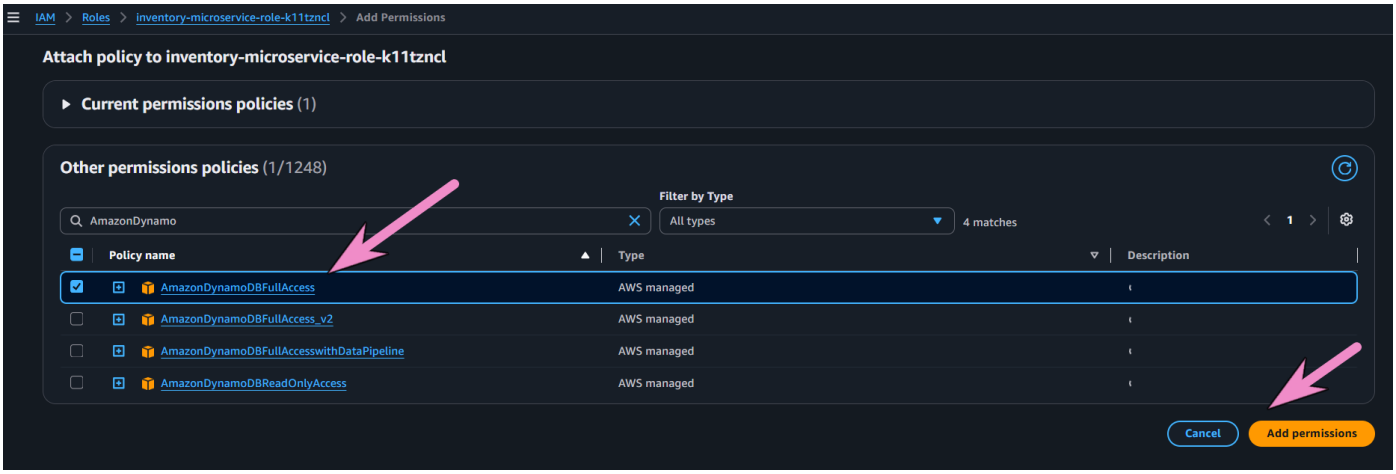
2. Click **Permissions** on the left menu.
3. Click the link under **Role name** (this opens AWS IAM in a new tab).



4. In the IAM tab, click **Add permissions** -> **Attach policies**.



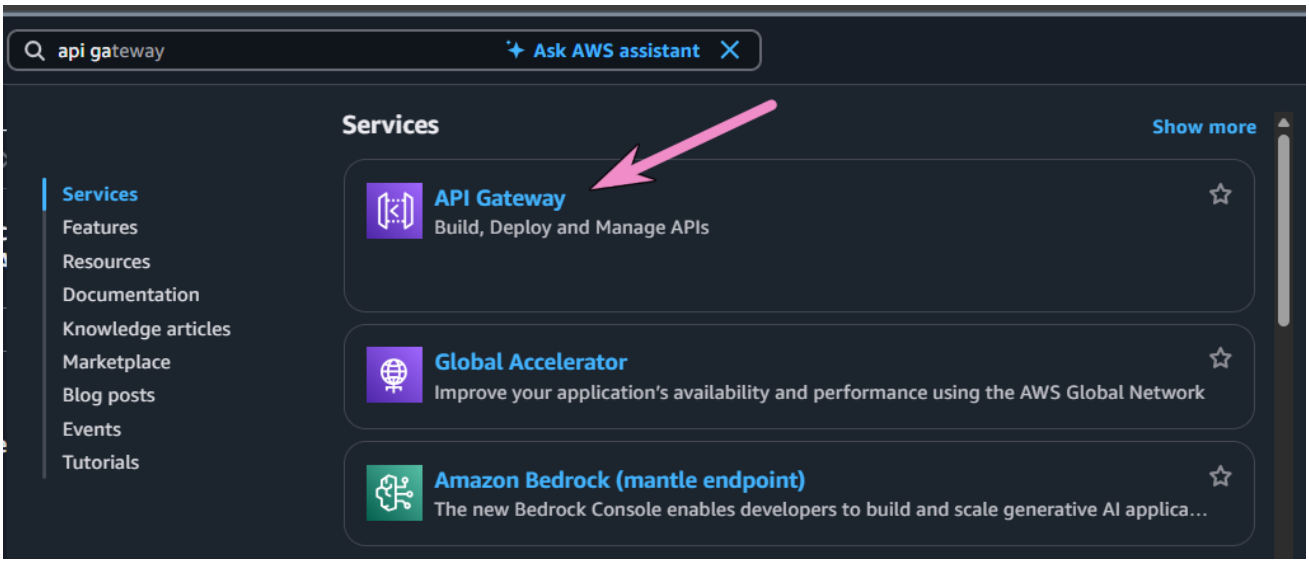
5. Search for AmazonDynamoDBFullAccess, check the box next to it, and click **Add permissions** at the bottom. You can now close this tab.



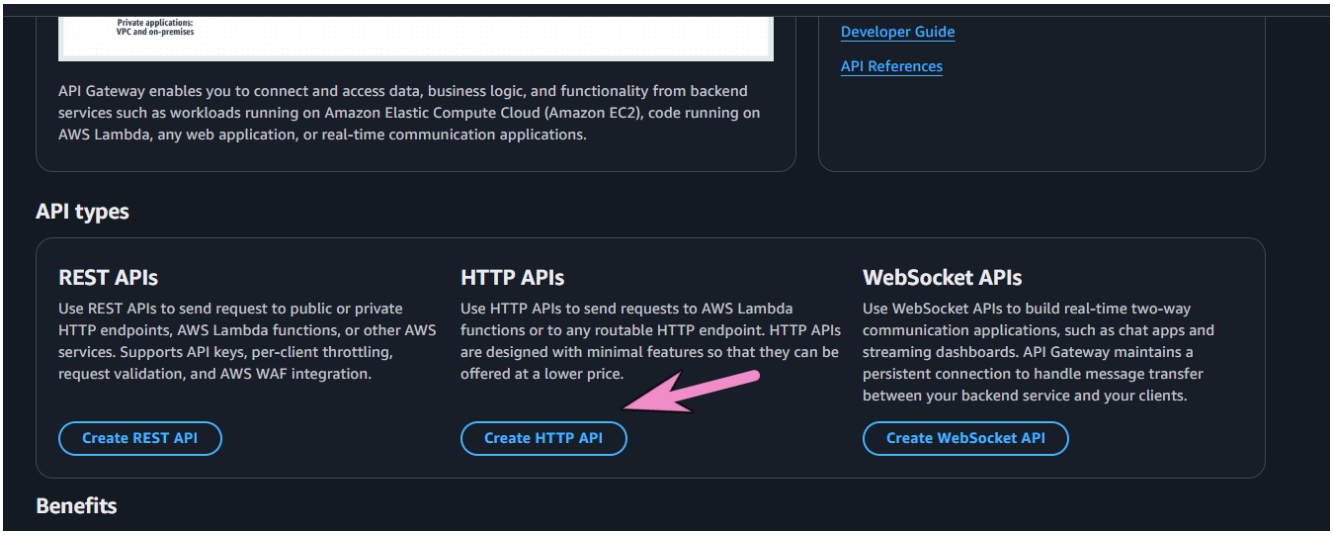
Step 3: Create the Gateway / Public URL (Amazon API Gateway)

Now we need a public URL so external applications or users can trigger our microservice. API Gateway gives you **1 million free HTTP API calls per month** for the first year.

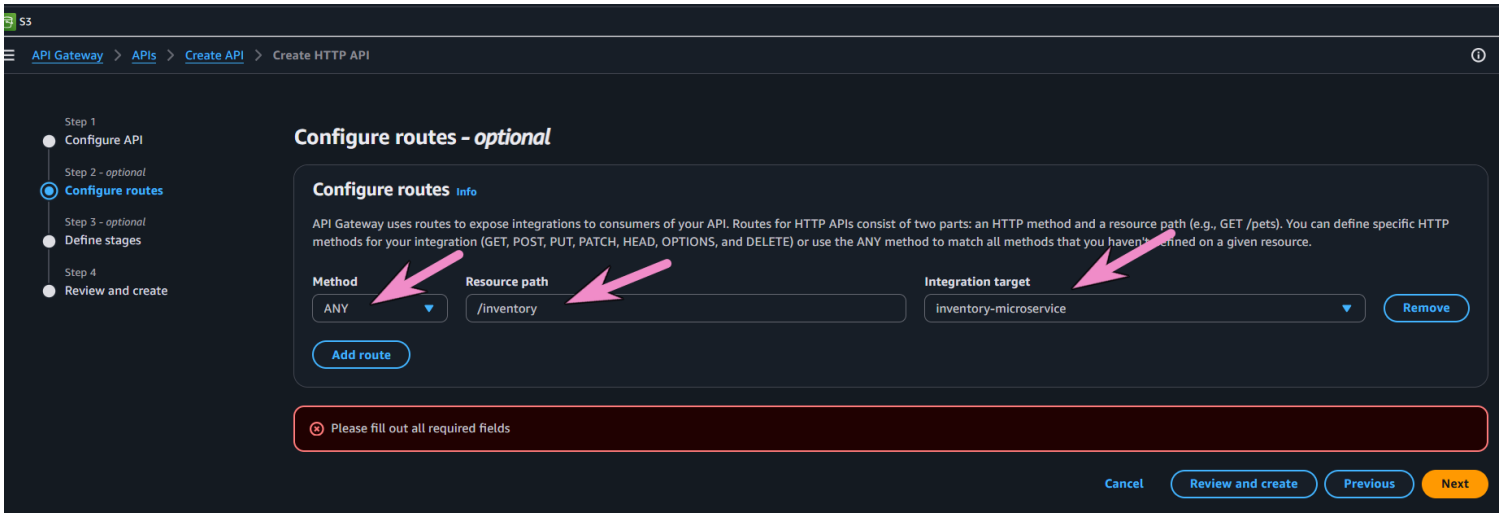
1. Search for **API Gateway** in the AWS Console search bar and open it.



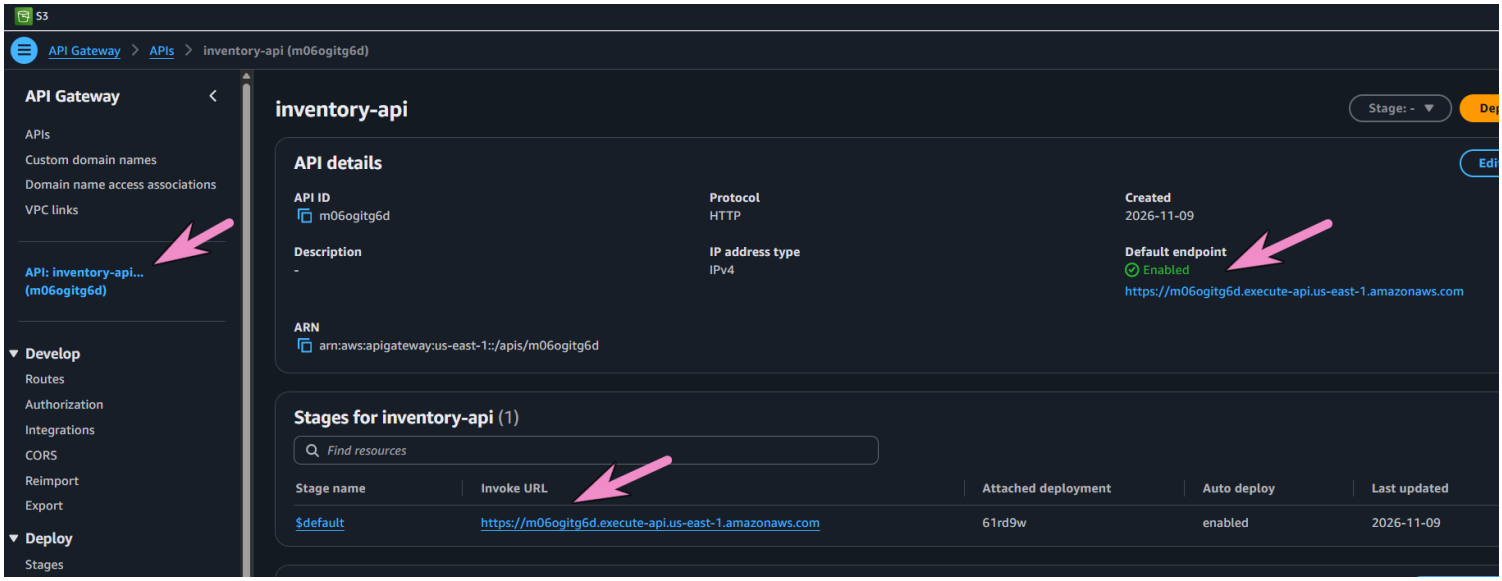
2. Look for **HTTP API** and click **Build** (HTTP APIs are faster and cheaper/free compared to REST APIs).



3. Configure the API:
    - **API name:** inventory-api
    - Click **Add integration** and choose **Lambda**.
    - Under **Lambda function**, select your inventory-microservice.
    - Click **Next**.
4. Configure Routes:
    - Change the Method to **ANY**.
    - Set the Resource path to /inventory.
    - Set Integration target to inventory-microservice
    - Click **Next**, then **Next** again on the stages page, and finally click **Create**.



5. Copy the **Invoke URL** displayed on your new API dashboard (it looks like https://amazonaws.com).



Step 4: Test Your Microservice

You now have a fully operational, independent microservice. You can test it using a free browser extension like **Talend API Tester**, a tool like **Postman**, or your computer's terminal command line.

Test 1: Add a product (POST)

Run this command in your terminal (replace YOUR_API_URL with the Invoke URL you copied):

curl.exe -X POST YOUR_API_URL/inventory -H "Content-Type: application/json" -d '{\"id\": \"101\", \"name\": \"Wireless Mouse\", \"price\": \"29.99\"}'

Expected response: {"message": "Product Wireless Mouse added successfully!"}

```
PS C:\WINDOWS\System32> curl.exe -X POST https://m06ogitg6d.execute-api.us-east-1.amazonaws.com/inventory
-H "Content-Type: application/json" -d '{\"id\": \"101\", \"name\": \"Wireless Mouse\", \"price\": \"29.99
\"}'
{"message": "Product Wireless Mouse added successfully!"}
PS C:\WINDOWS\System32>
```

Test 2: Retrieve the product (GET)

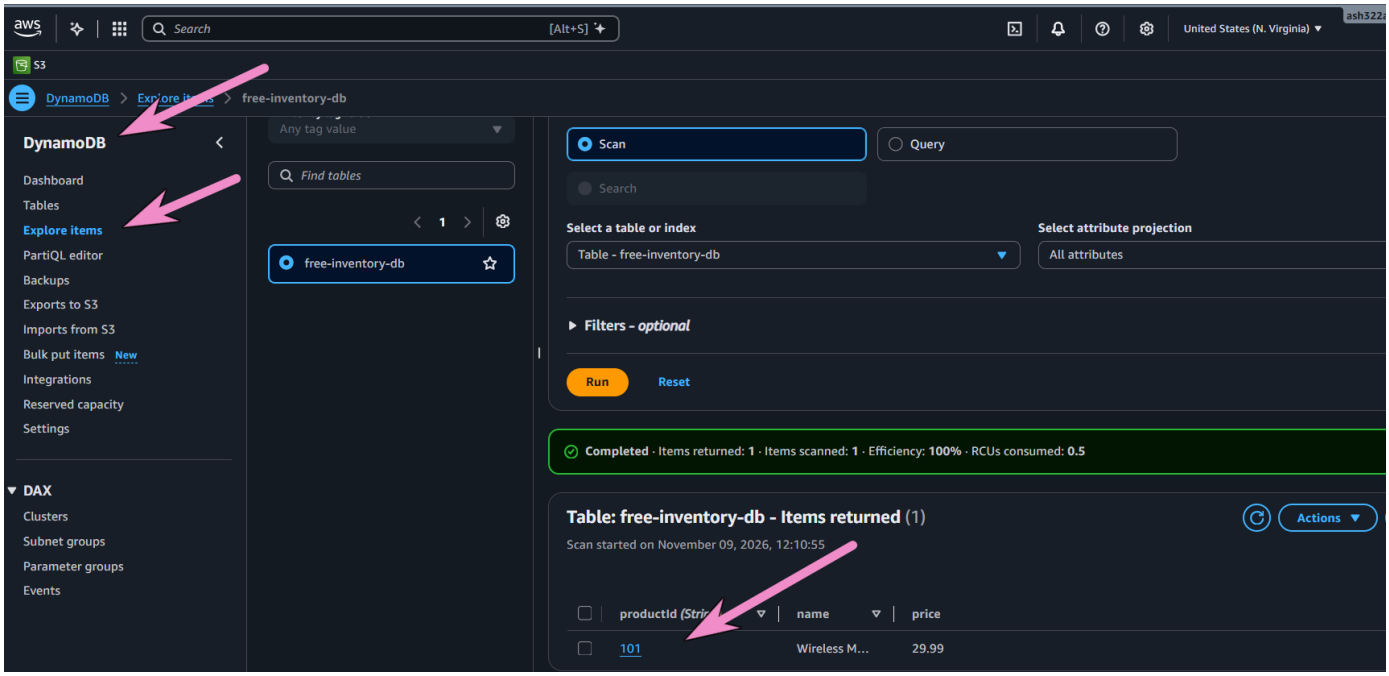
Paste this directly into your web browser or run it via terminal:

YOUR_API_URL/inventory?id=101



Expected response: {"productId": "101", "name": "Wireless Mouse", "price": "29.99"}

NOTE: If you open your DynamoDB table in the AWS Console and click **Explore table items**, you will see your wireless mouse successfully saved into the independent database cluster!



At the end delete all above services if you do not use them else cost may be incurred